



Pipex

Résumé : Ce projet vous permettra d'explorer en détail un mécanisme UNIX que vous connaissez déjà, en l'implémentant dans votre programme.

Version : 5.0

Sommaire

I	Avant-propos	2
II	Instructions générales	3
III	Instructions AI	5
IV	Partie obligatoire	7
IV.1	Exemples	8
IV.2	Conditions requises	8
V	Fichier Lisez-moi - Configuration requise	9
VI	Partie bonus	10
VII	Soumission et évaluation par les pairs	11

Chapitre I Avant- propos

Cristina : « Va danser la salsa ailleurs ! :) »

Chapitre II

Instructions générales

- Votre projet doit être écrit en C.
- Votre projet doit être rédigé conformément à la norme. Si vous avez inclus des fichiers ou des fonctions supplémentaires, ceux-ci sont pris en compte dans la vérification de conformité à la norme, et vous obtiendrez un 0 en cas d'erreur de conformité.
- Vos fonctions ne doivent pas se terminer de manière inattendue (erreur de segmentation, erreur de bus, double libération, etc.), sauf en cas de comportement indéfini. Si cela se produit, votre projet sera considéré comme non fonctionnel et recevra un 0 lors de l'évaluation.
- Toute la mémoire allouée sur le tas doit être correctement libérée lorsque cela est nécessaire. Les fuites de mémoire ne seront pas tolérées.
- Si le sujet l'exige, vous devez soumettre un Makefile qui compile vos fichiers source vers la sortie requise avec les drapeaux -Wall, -Wextra et -Werror, en utilisant cc. De plus, votre Makefile ne doit pas effectuer de liaison inutile.
- Votre Makefile doit contenir au moins les règles \$(NAME), all, clean, fclean et re.
- Pour soumettre des bonus pour votre projet, vous devez inclure une règle « bonus » dans votre Makefile, qui ajoutera tous les en-têtes, bibliothèques ou fonctions qui ne sont pas autorisés dans la partie principale du projet. Les bonus doivent être placés dans des fichiers _bonus.{c/h}, sauf indication contraire. L'évaluation des parties obligatoires et des bonus s'effectue séparément.
- Si votre projet vous permet d'utiliser votre libft, vous devez copier ses sources et son Makefile associé dans un dossier libft. Le Makefile de votre projet doit compiler la bibliothèque à l'aide de son Makefile, puis compiler le projet.
- Nous vous encourageons à créer des programmes de test pour votre projet, même si ce travail **n'a pas à être rendu et ne sera pas noté**. Cela vous permettra de tester facilement votre travail et celui de vos pairs. Ces tests vous seront particulièrement utiles lors de votre soutenance. En effet, lors de la soutenance, vous êtes libre d'utiliser vos tests et/ou ceux du pair que vous évaluez.
- Soumettez votre travail au dépôt Git qui vous a été attribué. Seul le travail présent dans le dépôt Git sera noté. Si Deepthought est chargé de noter votre travail, cela se fera

après vos évaluations par les pairs. Si une erreur survient dans une section quelconque de votre travail pendant la notation par Deepthought, l'évaluation s'arrêtera.

Chapitre III

Instructions pour l'IA

● Contexte

Au cours de votre parcours d'apprentissage, l'IA peut vous aider dans de nombreuses tâches différentes. Prenez le temps d'explorer les différentes capacités des outils d'IA et la manière dont ils peuvent vous aider dans votre travail. Cependant, abordez-les toujours avec prudence et évaluez les résultats de manière critique. Qu'il s'agisse de code, de documentation, d'idées ou d'explications techniques, vous ne pouvez jamais être tout à fait sûr que votre question était bien formulée ou que le contenu généré est exact. Vos pairs constituent une ressource précieuse pour vous aider à éviter les erreurs et les angles morts.

● Message principal

- ✚ Utilisez l'IA pour réduire les tâches répétitives ou fastidieuses.
- ✚ Développez des compétences en matière de formulation de requêtes — qu'elles impliquent ou non du codage — qui vous seront utiles dans votre future carrière.
- ✚ Apprenez comment fonctionnent les systèmes d'IA afin de mieux anticiper et éviter les risques courants, les biais et les problèmes éthiques.
- ✚ Continuez à développer vos compétences techniques et relationnelles en travaillant avec vos pairs.
- ✚ N'utilisez que du contenu généré par l'IA que vous comprenez parfaitement et dont vous pouvez assumer la responsabilité.

● Règles pour les apprenants :

- Vous devez prendre le temps d'explorer les outils d'IA et de comprendre leur fonctionnement, afin de pouvoir les utiliser de manière éthique et de réduire les biais potentiels.
- Avant de formuler votre demande, prenez le temps de réfléchir à votre problème : cela vous aidera à rédiger des demandes plus claires, plus détaillées et plus pertinentes, en utilisant un vocabulaire précis.
- Vous devriez prendre l'habitude de vérifier, de relire, de remettre en question et de tester systématiquement tout ce qui est généré par l'IA.
- Vous devriez toujours solliciter l'avis de vos pairs — ne vous fiez pas uniquement à votre propre validation.

● Résultats attendus :

- Développez des compétences en matière de formulation de requêtes, tant à usage général que spécifiques à un domaine.
- Améliorez votre productivité grâce à une utilisation efficace des outils d'IA.
- Continuer à renforcer la pensée computationnelle, la résolution de problèmes, l'adaptabilité et la collaboration.

● Commentaires et exemples :

- Vous serez régulièrement confronté à des situations — examens, évaluations, etc. — où vous devrez démontrer une réelle compréhension. Soyez prêt, continuez à développer vos compétences techniques et relationnelles.
- Expliquer votre raisonnement et débattre avec vos pairs permet souvent de mettre en évidence des lacunes dans votre compréhension. Faites de l'apprentissage entre pairs une priorité.
- Les outils d'IA ne disposent souvent pas de votre contexte spécifique et ont tendance à fournir des réponses génériques. Vos collègues, qui partagent votre environnement, peuvent vous apporter des informations plus pertinentes et plus précises.
- Alors que l'IA a tendance à générer la réponse la plus probable, vos pairs peuvent apporter des points de vue alternatifs et des nuances précieuses. Faites-les intervenir pour valider la qualité de vos conclusions.

✓ Bonne pratique :

Je demande à l'IA : « Comment tester une fonction de tri ? » Elle me donne quelques idées. Je les teste et examine les résultats avec un collègue. Nous affinons l'approche ensemble.

✗ Mauvaise pratique :

Je demande à l'IA d'écrire une fonction complète, puis je la copie-colle dans mon projet. Lors de l'évaluation par les pairs, je ne suis pas en mesure d'expliquer ce qu'elle fait ni pourquoi. Je perds toute crédibilité — et j'échoue dans mon projet.

✓ Bonne pratique :

J'utilise l'IA pour m'aider à concevoir un analyseur syntaxique. Ensuite, je passe en revue la logique avec un collègue. Nous repérons deux bugs et le réécrivons ensemble : le résultat est meilleur, plus clair et parfaitement compris.

✗ Mauvaise pratique :

Je laisse Copilot générer mon code pour une partie essentielle de mon projet. Il se compile, mais je ne peux pas expliquer comment il gère les pipes. Lors de l'évaluation, je ne parviens pas à justifier mon choix et j'échoue à mon projet.

Chapitre IV

Partie obligatoire

Nom du programme	pipex
Fichiers à soumettre	Makefile, *.h, *.c
Makefile	NAME, all, clean, fclean, re
Arguments	fichier1 commande1 commande2 fichier2
Fonction externe	• open, close, read, write, malloc, free, perror, strerror, access, dup, dup2, execve, exit, fork, pipe, unlink, wait, waitpid • ft_printf ou toute fonction équivalente que VOUS avez codée
Libft autorisée	Oui
Description	Ce projet porte sur la gestion des pipes.

Votre programme doit être exécuté comme suit :

```
./pipex fichier1 commande1 commande2 fichier2
```

Il doit prendre 4 arguments :

- fichier1 et fichier2 sont **des noms de fichiers**.
- cmd1 et cmd2 sont **des commandes shell** avec leurs paramètres.

Il doit se comporter exactement comme la commande shell suivante :

```
$> < fichier1 commande1 | commande2 > fichier2
```


IV.1 Exemples

```
$> ./pipex fichier_entrée "ls -l" "wc -l" fichier_sortie
```

Son comportement devrait être équivalent à : `< fichier_entrée ls -l | wc -l > fichier_sortie`

```
$> ./pipex infile "grep a1" "wc -w" outfile
```

Son comportement devrait être équivalent à : `< infile grep a1 | wc -w > outfile`

IV.2 Exigences

Votre projet doit respecter les règles suivantes :

- Vous devez fournir un Makefile qui compile vos fichiers source. Il ne doit pas effectuer de liaison inutile.
- Votre programme ne doit jamais se terminer de manière inattendue (par exemple, erreur de segmentation, erreur de bus, double libération, etc.).
- Votre programme ne doit pas présenter **de fuites de mémoire**.
- En cas de doute, gérez les erreurs de la même manière que la commande shell :
`< fichier1 cmd1 | cmd2 > fichier2`

Chapitre V

Exigences du fichier Lisez-moi

Un fichier README.md doit être placé à la racine de votre dépôt Git. Son objectif est de permettre à toute personne qui ne connaît pas le projet (collègues, membres du personnel, recruteurs, etc.) de comprendre rapidement en quoi consiste le projet, comment l'exécuter et où trouver davantage d'informations à ce sujet.

Le fichier README.md doit contenir au minimum :

- La toute première ligne doit être en italique et se lire : « *Ce projet a été créé dans le cadre du programme de formation de 42 par <login1>[, <login2>[, <login3>[...]]].* »
- Une section « Description » présentant clairement le projet, y compris son objectif et un bref aperçu.
- Une section « Instructions » contenant toutes les informations pertinentes concernant la compilation, l'installation et/ou l'exécution.
- Une section « Ressources » répertoriant les références classiques liées au sujet (documentation, articles, tutoriels, etc.), ainsi qu'une description de la manière dont l'IA a été utilisée — en précisant pour quelles tâches et quelles parties du projet.

► **Des sections supplémentaires peuvent être nécessaires en fonction du projet** (par exemple, exemples d'utilisation, liste des fonctionnalités, choix techniques, etc.).

Tout ajout requis sera explicitement indiqué ci-dessous.



L'anglais est recommandé ; vous pouvez également utiliser la langue principale de votre campus.

Chapitre VI

Partie bonus

Vous pouvez gagner des points supplémentaires si vous :

- Gérez plusieurs tuyaux.

Ceci :

```
$> ./pipex fichier1 commande1 commande2 commande3 ... commandeN fichier2
```

Devrait se comporter comme suit :

```
< fichier1 commande1 | commande2 | commande3 ... | commandeN > fichier2
```

- Prise en charge de « et » lorsque le premier paramètre est « here_doc ».

Ceci :

```
$> ./pipex here_doc LIMITER cmd cmd1 fichier
```

Devrait se comporter comme :

```
cmd << LIMITER | cmd1 >> file
```



La partie bonus ne sera évaluée que si la partie obligatoire est PARFAITE. Par « parfaite », on entend que la partie obligatoire a été intégralement réalisée et fonctionne sans dysfonctionnement. Si vous n'avez pas satisfait à TOUTES les exigences obligatoires, votre partie bonus ne sera pas évaluée du tout.

Chapitre VII

Soumission et évaluation par les pairs

Soumettez votre devoir dans votre dépôt Git comme d'habitude. Seul le travail contenu dans votre dépôt sera évalué lors de la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.

Au cours de l'évaluation, une brève **modification du projet** pourra parfois être demandée. Il peut s'agir d'un changement mineur de comportement, de quelques lignes de code à écrire ou à réécrire, ou d'une fonctionnalité facile à ajouter.

Même si cette étape **ne s'applique pas** nécessairement à **tous les projets**, vous devez vous y préparer si elle est mentionnée dans les directives d'évaluation.

Cette étape vise à vérifier votre compréhension réelle d'une partie spécifique du projet. La modification peut être effectuée dans l'environnement de développement de votre choix (par exemple, votre configuration habituelle) et devrait pouvoir être réalisée en quelques minutes — à moins qu'un délai spécifique ne soit défini dans le cadre de l'évaluation.

On peut, par exemple, vous demander d'apporter une petite mise à jour à une fonction ou à un script, de modifier un affichage ou d'ajuster une structure de données pour stocker de nouvelles informations, etc.

Les détails (portée, objectif, etc.) seront précisés dans les **directives d'évaluation** et peuvent varier d'une évaluation à l'autre pour un même projet.